

XRComm FCS 8T8R Platform

Application Development Guide

XRComm Inc.

June 2026

XRComm FCS 8T8R Platform

Application Development Guide

Platform	XRComm FCS 8T8R Platform
Driver Version	v1.4.3
Variant	8T8R
Audience	Software Developers
Classification	Proprietary

© 2026 XRComm Inc. – All Rights Reserved.

Contents

1	Introduction	3
2	Compiling and Running the Hello World Examples	3
2.1	Build the Hello-World Application	3
2.2	Run the Hello-World Application	3
2.3	Verify the Application Is Registered	4
3	Application API Reference	5
3.1	Quick API Use Case: Consume One 8T8R Channel	5
3.2	Connection & Lifecycle	5
3.2.1	xrcomm_ip_client_init_port_channel	5
3.2.2	xrcomm_ip_client_wait_ready	6
3.2.3	xrcomm_ip_client_running	6
3.2.4	xrcomm_ip_client_shutdown	6
3.3	Data Consumer Functions	6
3.3.1	xrcomm_ip_client_poll	6
3.3.2	xrcomm_ip_client_consume	6
3.4	Diagnostics	6
3.4.1	xrcomm_ip_client_get_dropped	6
3.4.2	xrcomm_ip_client_report_spectrum	6

1 Introduction

This document provides detailed guidelines for developers writing, compiling, and running custom applications that interface with the XRComm FCS 8T8R Platform.

The platform provides a header-only, zero-copy C API (`xrcomm_ip_client.h`) for POSIX applications that consume I/Q sample batches. Customer applications do not include DPDK or SPDK headers for IQ delivery; those libraries are used by the platform services. Reference documentation is available from DPDK (<https://core.dpdk.org/doc/>) and SPDK (<https://spdk.io/doc/>).

2 Compiling and Running the Hello World Examples

The platform includes an example consumer application in the `8T8R/hello_world/` folder that demonstrates how to subscribe to and consume platform data streams.

Example Binary	Delivery Mode	Enable With	Privileges
hello_world	IQ samples (zero-copy shared memory)	set-ip on	None

2.1 Build the Hello-World Application

Navigate to the hello world folder:

```
cd 8T8R/hello_world/
```

Compile using CMake:

```
mkdir -p build && cd build
cmake ..
make
```

Or compile directly with GCC:

```
gcc -O3 -mavx2 -pthread -Wall -o hello_world hello_world.c \
-I../platform/XRComm_platform_drivers/include \
-lrt -lpthread -lm
```

Compiled binaries are placed in `bin/` (if compiled with CMake) or the folder root (if compiled with GCC).

2.2 Run the Hello-World Application

The platform driver must be running and IQ mode must be enabled before starting the application:

Terminal 1 (FlexCompute Server): From the `XRComm_FCS_platform` root, ensure `xrcomm_server` is running, then enable IQ mode:

```
python3 8T8R/platform/XRComm_platform_gRPC/client/xrcomm_grpc_client.py \
--host 127.0.0.1 --port 50051 set-ip on
```

Terminal 2 (FlexCompute Server): Run the application:

```
./8T8R/hello_world/bin/hello_world
```

The example prints a summary of batches, samples, signal power, and drops for all 8 channels:

```
[IP:hello_world_ch0] Registered slot 0 (pid 12345, port 0, ch 0)
...
[HW] All channels ready – starting receive loop
[HW] Channel summary (port 0):
  ch0: batches=15234 samples=487488 power=-18.4dBFS mean=-18.6dBFS dropped=0
```

```

ch1: batches=15234 samples=487488 power=-19.1dBFS mean=-19.2dBFS dropped=0
ch2: batches=15234 samples=487488 power=-18.9dBFS mean=-19.0dBFS dropped=0
ch3: batches=15234 samples=487488 power=-18.7dBFS mean=-18.8dBFS dropped=0
[HW] Channel summary (port 1):
ch4: batches=15234 samples=487488 power=-19.3dBFS mean=-19.4dBFS dropped=0
ch5: batches=15234 samples=487488 power=-18.8dBFS mean=-18.9dBFS dropped=0
ch6: batches=15234 samples=487488 power=-19.0dBFS mean=-19.1dBFS dropped=0
ch7: batches=15234 samples=487488 power=-18.6dBFS mean=-18.7dBFS dropped=0

```

2.3 Verify the Application Is Registered

Query the registry from the **Client Computer**:

```
python3 xrcomm_grpc_client.py --host <server-ip> --port 50051 list-ips
```

Each of the 8 channel slots should appear active:

```

slot=0 name=hello_world_ch0 pid=12345 mode=IQ_MODE port=0 ch=0 ring=ready active=yes
slot=1 name=hello_world_ch1 pid=12345 mode=IQ_MODE port=0 ch=1 ring=ready active=yes
slot=2 name=hello_world_ch2 pid=12345 mode=IQ_MODE port=0 ch=2 ring=ready active=yes
slot=3 name=hello_world_ch3 pid=12345 mode=IQ_MODE port=0 ch=3 ring=ready active=yes
slot=4 name=hello_world_ch4 pid=12345 mode=IQ_MODE port=1 ch=4 ring=ready active=yes
slot=5 name=hello_world_ch5 pid=12345 mode=IQ_MODE port=1 ch=5 ring=ready active=yes
slot=6 name=hello_world_ch6 pid=12345 mode=IQ_MODE port=1 ch=6 ring=ready active=yes
slot=7 name=hello_world_ch7 pid=12345 mode=IQ_MODE port=1 ch=7 ring=ready active=yes

```

3 Application API Reference

Include `xrcomm_ip_client.h` (found in `XRComm_platform_drivers/include/`) in your applications. No DPDK headers are required.

3.1 Quick API Use Case: Consume One 8T8R Channel

The typical customer application opens one logical channel, waits until the platform has created the shared-memory ring, then polls batches until the platform stops:

```
#include <stdio.h>
#include "xrcomm_ip_client.h"

int main(void) {
    xrcomm_ip_client_t client;
    if (xrcomm_ip_client_init_port_channel(&client, "my_ch0_app", 0, 0) != 0) {
        return 1;
    }
    if (xrcomm_ip_client_wait_ready(&client, 5000) != 0) {
        xrcomm_ip_client_shutdown(&client);
        return 1;
    }

    while (xrcomm_ip_client_running(&client)) {
        const xrcomm_iq_slot_t *slot = xrcomm_ip_client_poll(&client);
        if (!slot) {
            continue;
        }

        printf("port=%u channel=%u samples=%u\n",
            slot->hsp_port, slot->channel, slot->valid_sample_count);
        xrcomm_ip_client_report_spectrum(&client, 1, 0.0f);
        xrcomm_ip_client_consume(&client);
    }

    xrcomm_ip_client_shutdown(&client);
    return 0;
}
```

Run one instance per channel when you want separate per-channel processing, or follow the hello-world pattern to register all 8 channels from one process.

3.2 Connection & Lifecycle

3.2.1 `xrcomm_ip_client_init_port_channel`

```
int xrcomm_ip_client_init_port_channel(xrcomm_ip_client_t *client,
    const char *name,
    uint32_t hsp_port,
    uint32_t channel);
```

8T8R: Registers the application to consume the IQ stream of a single channel of one HSP port. Arguments: client handle, application name (max 63 chars), hsp_port (0 or 1), channel (0–7). Returns 0 on success.

3.2.2 xrcomm_ip_client_wait_ready

```
int xrcomm_ip_client_wait_ready(xrcomm_ip_client_t *client, int timeout_ms);
```

Blocks the caller until the platform has created and opened the shared memory data ring for this channel. Timeout of 0 means wait indefinitely. Returns 0 when ready, -1 on timeout.

3.2.3 xrcomm_ip_client_running

```
bool xrcomm_ip_client_running(xrcomm_ip_client_t *client);
```

Returns true as long as the platform's receive loop is active. Use this as the main condition for your ingest loop.

3.2.4 xrcomm_ip_client_shutdown

```
void xrcomm_ip_client_shutdown(xrcomm_ip_client_t *client);
```

Deregisters the channel handle from the platform and releases all mapped shared memory regions.

3.3 Data Consumer Functions

3.3.1 xrcomm_ip_client_poll

```
const xrcomm_iq_slot_t* xrcomm_ip_client_poll(xrcomm_ip_client_t *client);
```

Polls the data ring for a new batch of IQ samples. Returns a read-only pointer to the batch if available, or NULL if empty or if IQ mode is off. Batch fields: - samples[]: Array of interleaved little-endian int16 IQ pairs - valid_sample_count: Count of valid samples in the batch - hsp_port, channel: Metadata indicating source port and channel

3.3.2 xrcomm_ip_client_consume

```
void xrcomm_ip_client_consume(xrcomm_ip_client_t *client);
```

Releases the current batch slot and returns it to the platform's write pool. Must be called once per successful poll.

3.4 Diagnostics

3.4.1 xrcomm_ip_client_get_dropped

```
uint64_t xrcomm_ip_client_get_dropped(xrcomm_ip_client_t *client);
```

Returns the number of batches dropped by the platform for this subscription because the application polled too slowly.

3.4.2 xrcomm_ip_client_report_spectrum

```
void xrcomm_ip_client_report_spectrum(xrcomm_ip_client_t *client,  
                                     uint64_t processed,  
                                     float power_dbfs);
```

Reports processing stats back to the platform, making them visible in gRPC telemetry and statistics.